

ASSIGNMENT – 6

Simulation of UDP Packet Transmission Using SimPy (Smart-City Surveillance System)

AIM

To simulate UDP packet transmission in a smart-city surveillance system using SimPy. Roadside cameras generate video packets and send them to a central monitoring server over UDP; since UDP does not guarantee delivery, packets may be lost due to congestion and experience random transmission delays. The simulation models packet generation, transmission, random delay, and packet loss, and analyzes the performance of UDP communication under different network conditions.

The UDP Client module accepts the number of packets, packet size, generation interval, and packet-loss probability; it generates packets periodically with a SimPy process, assigns a unique sequence number and generation timestamp to each packet, applies a random transmission delay, decides randomly whether the packet is transmitted or lost, and puts transmitted packets on a shared SimPy communication channel. The UDP Server module receives the packets, records arrival times, computes per-packet delay, detects missing packets from sequence numbers, and maintains statistics such as packets received, packets lost, packet delivery ratio, packet loss ratio, and minimum/maximum/average delay. Four network conditions are simulated: error-free (0% loss), low loss (10%), moderate loss (20%), and high loss (30%).

ALGORITHM

- CLIENT: every generation interval, create a packet with sequence number and generation timestamp.
- CLIENT: with probability equal to the loss probability, mark the packet as lost.
- CLIENT: otherwise wait a random transmission delay and put the packet on the shared channel.
- CLIENT: count generated, transmitted, and lost packets.
- SERVER: take each packet from the channel, record the arrival time and compute the delay from the generation timestamp.
- SERVER: at the end, detect missing packets from sequence numbers and print loss ratio, delivery ratio, and delay statistics.

UDP CLIENT MODULE - `udp_client.py`

```
import random

# udp client simulation module: generates packets periodically with a simpy
# process, applies a random transmission delay, and loses packets based on
# the loss probability before putting them on the communication channel

class UDPClient:
```

```
def __init__(self, env, channel, packets, size, interval, loss_prob,
              delay_min, delay_max, out, detail):
    self.env = env
    self.channel = channel
    self.packets = packets
    self.size = size
    self.interval = interval
    self.loss_prob = loss_prob
    self.delay_min = delay_min
    self.delay_max = delay_max
    self.out = out
    self.detail = detail
    self.generated = 0
    self.transmitted = 0
    self.lost = 0

def run(self):
    o = self.out
    print("UDP CLIENT SIMULATION", file=o)
    print(f"Packets to Generate : {self.packets}", file=o)
    print(f"Packet Size          : {self.size} bytes", file=o)
    print(f"Generation Interval : {int(self.interval * 1000)} ms", file=o)
    print(f"Loss Probability      : {int(self.loss_prob * 100)}%", file=o)
    print(f"Delay Range           : {int(self.delay_min * 1000)}-{int(self.delay_max * 1000)} ms", file=o)
    print(file=o)
    for seq in range(1, self.packets + 1):
        yield self.env.timeout(self.interval)
        gen_time = self.env.now
        if self.detail:
            print(f"Packet {seq} Generated at {gen_time:.2f} s", file=o)
        self.generated += 1
        if random.random() < self.loss_prob:
            # packet is lost based on the loss probability
            if self.detail:
                print(f"Packet {seq} Lost", file=o)
            self.lost += 1
            continue
        # random transmission delay before the packet reaches the channel
        delay = random.uniform(self.delay_min, self.delay_max)
        yield self.env.timeout(delay)
        if self.detail:
            print(f"Packet {seq} Transmitted", file=o)
        self.transmitted += 1
        yield self.channel.put((seq, gen_time, self.size))
        if self.detail:
            print(file=o)
    print(f"Total Packets Generated   : {self.generated}", file=o)
    print(f"Total Packets Transmitted : {self.transmitted}", file=o)
    print(f"Total Packets Lost          : {self.lost}", file=o)
    print(file=o)
```

UDP SERVER MODULE - udp_server.py

```
# udp server simulation module: receives packets from the channel, records
# arrival times, computes the delay, and reports statistics for the run

class UDPServer:
    def __init__(self, env, channel, packets, out, detail):
        self.env = env
        self.channel = channel
        self.packets = packets
        self.out = out
        self.detail = detail
        self.received = []
        self.delays = []
        self.expected = 1

    def run(self):
        o = self.out
        print("UDP SERVER SIMULATION", file=o)
        print(f"Total Packets Expected : {self.packets}", file=o)
        print(file=o)
        while True:
            seq, gen_time, size = yield self.channel.get()
            arrival = self.env.now
            delay = arrival - gen_time
            self.received.append(seq)
            self.delays.append(delay)
            if self.detail:
                print(f"Packet {seq} Received", file=o)
                print(f"Generation Time : {gen_time:.2f} s", file=o)
                print(f"Arrival Time      : {arrival:.2f} s", file=o)
                print(f"Delay              : {delay:.2f} s", file=o)
                print(file=o)

    def summary(self):
        o = self.out
        received_count = len(self.received)
        lost = self.packets - received_count
        print("Packets Received      :", received_count, file=o)
        print("Packets Lost           :", lost, file=o)
        print("Packet Loss Ratio       :", f"{lost / self.packets * 100:.1f}%", file=o)
        print("Packet Delivery Ratio   :", f"{received_count / self.packets * 100:.1f}%", file=o)
        if self.delays:
            print("Minimum Delay          :", f"{min(self.delays):.2f} s", file=o)
            print("Maximum Delay          :", f"{max(self.delays):.2f} s", file=o)
            print("Average Delay          :", f"{sum(self.delays) / len(self.delays):.2f} s", file=o)
        # detect missing packets from the sequence numbers
        missing = [s for s in range(1, self.packets + 1) if s not in self.received]
        if missing:
            print("Missing Packets        :", " ".join(map(str, missing)), file=o)
        print(file=o)
```

CLIENT OUTPUT (FULL TRACE FOR TC2 + SUMMARIES)

```
===== TC1 =====  
UDP CLIENT SIMULATION  
Packets to Generate : 10  
Packet Size          : 100 bytes  
Generation Interval : 200 ms  
Loss Probability     : 0%  
Delay Range          : 0-0 ms
```

```
Total Packets Generated : 10  
Total Packets Transmitted : 10  
Total Packets Lost      : 0
```

```
===== TC2 =====  
UDP CLIENT SIMULATION  
Packets to Generate : 20  
Packet Size          : 100 bytes  
Generation Interval : 200 ms  
Loss Probability     : 10%  
Delay Range          : 10-50 ms
```

```
Packet 1 Generated at 0.20 s  
Packet 1 Transmitted
```

```
Packet 2 Generated at 0.43 s  
Packet 2 Transmitted
```

```
Packet 3 Generated at 0.64 s  
Packet 3 Lost  
Packet 4 Generated at 0.84 s  
Packet 4 Transmitted
```

```
Packet 5 Generated at 1.07 s  
Packet 5 Lost  
Packet 6 Generated at 1.27 s  
Packet 6 Transmitted
```

```
Packet 7 Generated at 1.52 s  
Packet 7 Transmitted
```

```
Packet 8 Generated at 1.74 s  
Packet 8 Transmitted
```

```
Packet 9 Generated at 1.96 s  
Packet 9 Transmitted
```

```
Packet 10 Generated at 2.18 s  
Packet 10 Transmitted
```

```
Packet 11 Generated at 2.39 s
```

Packet 11 Transmitted

Packet 12 Generated at 2.64 s

Packet 12 Transmitted

Packet 13 Generated at 2.86 s

Packet 13 Transmitted

Packet 14 Generated at 3.09 s

Packet 14 Transmitted

Packet 15 Generated at 3.30 s

Packet 15 Transmitted

Packet 16 Generated at 3.55 s

Packet 16 Transmitted

Packet 17 Generated at 3.79 s

Packet 17 Transmitted

Packet 18 Generated at 4.01 s

Packet 18 Transmitted

Packet 19 Generated at 4.25 s

Packet 19 Transmitted

Packet 20 Generated at 4.47 s

Packet 20 Transmitted

Total Packets Generated : 20

Total Packets Transmitted : 18

Total Packets Lost : 2

===== TC3 =====

UDP CLIENT SIMULATION

Packets to Generate : 30

Packet Size : 200 bytes

Generation Interval : 100 ms

Loss Probability : 20%

Delay Range : 20-100 ms

Total Packets Generated : 30

Total Packets Transmitted : 24

Total Packets Lost : 6

===== TC4 =====

UDP CLIENT SIMULATION

Packets to Generate : 50

Packet Size : 200 bytes

Generation Interval : 67 ms

Loss Probability : 30%

Delay Range : 50-150 ms

Total Packets Generated : 50

Total Packets Transmitted : 35

Total Packets Lost : 15

SERVER OUTPUT (FULL TRACE FOR TC2 + SUMMARIES)

===== TC1 =====

UDP SERVER SIMULATION

Total Packets Expected : 10

Packets Received : 10

Packets Lost : 0

Packet Loss Ratio : 0.0%

Packet Delivery Ratio : 100.0%

Minimum Delay : 0.00 s

Maximum Delay : 0.00 s

Average Delay : 0.00 s

===== TC2 =====

UDP SERVER SIMULATION

Total Packets Expected : 20

Packet 1 Received

Generation Time : 0.20 s

Arrival Time : 0.23 s

Delay : 0.03 s

Packet 2 Received

Generation Time : 0.43 s

Arrival Time : 0.44 s

Delay : 0.01 s

Packet 4 Received

Generation Time : 0.84 s

Arrival Time : 0.87 s

Delay : 0.03 s

Packet 6 Received

Generation Time : 1.27 s

Arrival Time : 1.32 s

Delay : 0.05 s

Packet 7 Received

Generation Time : 1.52 s

Arrival Time : 1.54 s

Delay : 0.03 s

Packet 8 Received

Generation Time : 1.74 s

Arrival Time : 1.76 s

Delay : 0.02 s

Packet 9 Received

Generation Time : 1.96 s

Arrival Time : 1.98 s

Delay : 0.02 s

Packet 10 Received

Generation Time : 2.18 s

Arrival Time : 2.19 s

Delay : 0.01 s

Packet 11 Received

Generation Time : 2.39 s

Arrival Time : 2.44 s

Delay : 0.04 s

Packet 12 Received

Generation Time : 2.64 s

Arrival Time : 2.66 s

Delay : 0.03 s

Packet 13 Received

Generation Time : 2.86 s

Arrival Time : 2.89 s

Delay : 0.03 s

Packet 14 Received

Generation Time : 3.09 s

Arrival Time : 3.10 s

Delay : 0.01 s

Packet 15 Received

Generation Time : 3.30 s

Arrival Time : 3.35 s

Delay : 0.05 s

Packet 16 Received

Generation Time : 3.55 s

Arrival Time : 3.59 s

Delay : 0.04 s

Packet 17 Received

Generation Time : 3.79 s

Arrival Time : 3.81 s

Delay : 0.02 s

Packet 18 Received

Generation Time : 4.01 s
Arrival Time : 4.05 s
Delay : 0.04 s

Packet 19 Received

Generation Time : 4.25 s
Arrival Time : 4.27 s
Delay : 0.02 s

Packet 20 Received

Generation Time : 4.47 s
Arrival Time : 4.50 s
Delay : 0.03 s

Packets Received : 18
Packets Lost : 2
Packet Loss Ratio : 10.0%
Packet Delivery Ratio : 90.0%
Minimum Delay : 0.01 s
Maximum Delay : 0.05 s
Average Delay : 0.03 s
Missing Packets : 3 5

===== TC3 =====

UDP SERVER SIMULATION

Total Packets Expected : 30

Packets Received : 24
Packets Lost : 6
Packet Loss Ratio : 20.0%
Packet Delivery Ratio : 80.0%
Minimum Delay : 0.02 s
Maximum Delay : 0.10 s
Average Delay : 0.06 s
Missing Packets : 2 16 18 21 22 25

===== TC4 =====

UDP SERVER SIMULATION

Total Packets Expected : 50

Packets Received : 35
Packets Lost : 15
Packet Loss Ratio : 30.0%
Packet Delivery Ratio : 70.0%
Minimum Delay : 0.06 s
Maximum Delay : 0.15 s
Average Delay : 0.10 s
Missing Packets : 2 6 7 8 10 12 21 26 30 35 39 41 43 45 47

PERFORMANCE COMPARISON

Test Case	Configured Loss	Generated	Transmitted	Received	Lost	Loss Ratio	Delivery Ratio	Avg Delay
TC1	0%	10	10	10	0	0.0%	100.0%	0.00 s
TC2	10%	20	18	18	2	10.0%	90.0%	0.03 s
TC3	20%	30	24	24	6	20.0%	80.0%	0.06 s
TC4	30%	50	35	35	15	30.0%	70.0%	0.10 s

TASKS PERFORMED

Task	Details	Result
SimPy processes	Client and server implemented as SimPy process generators sharing a Store channel	All four scenarios ran end to end
Packet generation	Packets generated every interval with sequence number and generation timestamp	10, 20, 30 and 50 packets generated per scenario
Random delay and loss	Uniform transmission delay in the configured range; random loss by probability	Delays up to 0.15 s and losses of 0, 2, 6, 15 packets observed
Reception statistics	Arrival time and delay computed for every received packet	Average delay grew from 0.00 s to 0.10 s across scenarios
Missing detection	Missing packets identified from sequence number gaps	Missing lists printed, e.g. packets 3 and 5 in TC2
Rate effect	Higher loss probability with faster generation and bigger delays	Delivery ratio dropped from 100% to 70%

LEARNING OUTCOMES

- I modelled packet generation, transmission delay, and packet loss with SimPy processes.
- I used a SimPy Store as the communication channel between client and server.
- I computed delivery ratio, loss ratio, and delay statistics from real runs.
- I detected missing packets from sequence number gaps.
- I observed that higher loss probability and faster generation reduce the delivery ratio.